

Entropy-Guided Retrieval and Layer-2 Semantic Reuse for Retrieval-Augmented Generation

Yunus Günay

Computer Science

yunus.gunay@ug.bilkent.edu.tr

Abstract—Retrieval-Augmented Generation (RAG) improves factuality by grounding large language models (LLMs) on external evidence, but it also increases inference cost by appending retrieved passages to the input, inflating the prompt-processing (prefill) stage and Time-To-First-Token (TTFT). This paper presents a system-level simulator that studies how retrieval-side uncertainty estimation and reuse-aware execution can improve RAG efficiency without modifying the underlying language model. We introduce entropy-guided adaptive retrieval, which estimates retriever confidence directly from the similarity-score distribution of a dense FAISS retriever using temperature-scaled softmax and normalized Shannon entropy. This uncertainty signal dynamically controls retrieval depth on a per-query basis, reducing unnecessary context when retrieval is confident. In addition, we propose a two-layer reuse mechanism to reduce redundant prefill computation: Layer-1 simulates RAGCache-style exact-prefix KV reuse over ordered retrieval results, while Layer-2 introduces semantic reuse that enables partial reuse when retrieved contexts are semantically similar but not identical. Across realistic online query patterns, exact-prefix reuse achieves up to 90% TTFT reduction when retrieval results repeat, while semantic reuse recovers 50% TTFT reduction in rephrased-query scenarios where exact reuse fails. Entropy-guided retrieval further reduces retrieved token budget by up to 25.3% with only a modest drop in an accuracy proxy. Overall, the results demonstrate that retriever-side uncertainty signals and semantic similarity can be combined to improve interactive RAG serving efficiency under realistic queries.

Index Terms—Retrieval-Augmented Generation, Adaptive Retrieval, Entropy, Uncertainty Estimation, Key-Value Cache, Prefix Reuse, Semantic Caching

I. INTRODUCTION

A. Background

Large Language Models (LLMs) are the backbone of many modern natural language processing systems, including question answering, reasoning, and text generation. Their recent progress is enabled by the Transformer architecture, which replaces recurrence with self-attention mechanisms [1]. This design allows models to capture dependencies between tokens regardless of their distance in the input or output sequence [1].

Despite their strong generative capabilities, LLMs primarily rely on parametric (internal) knowledge acquired during training, which may be outdated, incomplete, or unreliable for knowledge-intensive queries [2]. The reliability of model outputs therefore depends on whether the required information is encoded in the model parameters or must be supported externally. This limitation motivates the use of mechanisms that improve factual robustness and grounding. Retrieval-Augmented Generation (RAG) [3] addresses this limitation by

coupling a retriever with a generator. For each query, relevant passages are retrieved from a large external corpus, and the generator conditions its output on this retrieved evidence [4]. By incorporating non-parametric knowledge, RAG improves response accuracy and allows knowledge updates without retraining the underlying language model.

However, these benefits come with non-trivial efficiency and scalability costs. A complete RAG pipeline involves corpus preprocessing, index construction, real-time retrieval, and generation, all of which introduce computational overhead. These costs become especially pronounced at scale, where relevant information may be sparsely distributed across large and heterogeneous corpora [5]. Furthermore, even when retrieval is effective, the amount of retrieved text that can be provided to the generator is constrained by the finite context window of the model, limiting how much external evidence can be injected per request [5].

From a serving perspective, inference latency is critically influenced by the prompt-processing (prefill) phase, during which the model consumes the entire input prompt to compute attention key-value (KV) states before producing the first output token [6]. The latency incurred in this phase is commonly measured as Time-To-First-Token (TTFT), a key performance metric for interactive LLM systems [7]. Since RAG augments the prompt with retrieved passages, it directly increases prompt length and consequently, prefill cost and TTFT. In addition, longer prompts require storing larger KV caches, and memory consumption grows approximately linearly with the number of input tokens [8].

These observations motivate this work. Specifically, we seek to (i) introduce semantic awareness into retrieval decisions to reduce unnecessary retrieved context, and (ii) enable reuse opportunities that reduce redundant prompt-processing computation. Importantly, our approach operates entirely on the system level. It does not modify the architecture, parameters, or inference behavior of the underlying language model. Instead, it leverages retriever-side uncertainty estimation and semantic similarity to improve computational efficiency (TTFT) within standard dense-retrieval and Transformer-based inference pipelines.

B. Methodological Overview

A recently proposed system, RAGCache [9], reduces inference latency in RAG pipelines by reusing previously computed attention key-value (KV) tensors during the prompt prefill

phase. When the same retrieved documents appear again in the same order, the corresponding KV states can be reused, substantially reducing TTFT [9]. However, this reuse mechanism is limited to exact-prefix matches of retrieved document sequences, making it fragile under realistic query streams where even small variations in retrieval order or composition are common. In practice, users often issue semantically similar but lexically different queries, such as “Who teaches NLP?” versus “Which person is the teacher of NLP?” which can alter retrieval rankings and invalidate exact-prefix reuse, leading to redundant computation.

Motivated by this limitation, the proposed approach is guided by two central research questions:

- **Adaptive Retrieval Efficiency:** How can the retrieval depth in Retrieval-Augmented Generation systems be dynamically adjusted to reduce unnecessary retrieved context and generation-time computation, while preserving answer quality as much as possible?
- **Semantic Reuse Beyond Exact Matching:** When exact-prefix cache reuse is not possible due to small variations in retrieved contexts, can semantic similarity be leveraged to enable approximate reuse of previously computed intermediate representations and thereby reduce redundant prefill computation?

To address these questions, we design a lightweight, system-level approach that combines entropy-guided adaptive retrieval with a two-layer reuse mechanism. We evaluate this framework on the TriviaQA [10] dataset, which is widely used in RAG research. After preprocessing, documents are encoded using the all-MiniLM-L6-v2 sentence transformer model, which maps passages into a 384-dimensional dense vector space [11]. These embeddings are indexed using FAISS (Facebook AI Similarity Search) [12], an efficient open-source library for large-scale similarity search over dense vector representations [13].

For each query, dense retrieval produces similarity scores over the top- K candidate passages. A normalized Shannon entropy is computed from this score distribution to estimate retriever uncertainty, and the number of retrieved passages is adjusted accordingly. Lower entropy indicates higher confidence and leads to fewer retrieved passages, while higher entropy triggers broader retrieval [14]. Following retrieval, we simulate two reuse stages during the prompt prefill phase. Layer-1 implements exact-prefix reuse inspired by RAGCache [9]. When Layer-1 reuse is not possible, Layer-2 performs semantic reuse by identifying previously observed retrieved contexts that are semantically similar to the current one and modeling partial reuse of prefill computation.

Crucially, both caching layers are implemented as a simulation of prefill cost reduction rather than as a modification of the language model itself. The exact construction of the final prompt, the attention computation, and the internal inference dynamics of the LLM are treated as a black box. Instead, the system estimates potential TTFT reductions that could be achieved if semantic-level reuse were supported by a production-grade serving system. This abstraction allows

us to isolate and study the efficiency gains introduced by uncertainty-aware retrieval and semantic reuse, independent of hardware-specific optimizations or model-specific implementation details.

C. Expected Outcomes

The proposed approach is expected to improve the efficiency of Retrieval-Augmented Generation systems along two complementary dimensions. First, entropy-guided adaptive retriever is expected to reduce the amount of retrieved context on a per-query basis by dynamically adjusting retrieval depth, thereby lowering prompt length while largely preserving evidence coverage. Second, the two-layer reuse mechanism is expected to significantly reduce Time-To-First-Token (TTFT) by avoiding redundant prompt-prefill computation, even in cases where exact retrieval repetition does not occur.

D. Report Organization

The remainder of this report is organized as follows. Section II reviews related work on uncertainty-aware retrieval, entropy-based retrieval strategies, and KV-cache reuse mechanisms for efficient RAG serving. Section III presents the proposed system architecture and experimental methodology, including data preprocessing, dense index construction, entropy calibration for adaptive retrieval, and the Layer-1 & Layer-2 reuse simulations. Section IV reports experimental results under multiple interactive query patterns that exercise different reuse paths and uncertainty regimes. Finally, Section V concludes the paper with a summary of key findings and directions for future work.

II. RELATED WORK

A. Adaptive Retrieval Based on Uncertainty Estimation

Traditional RAG pipelines retrieve a fixed top- k set of passages for each query, regardless of the query difficulty or retriever confidence. This static strategy can introduce redundant or weakly relevant evidence, which may degrade answer quality while increasing computational cost. Consequently, recent work has explored adaptive retrieval mechanisms that condition retrieval behavior on uncertainty signals. Self-RAG [16], for example, enables the language model to decide when retrieval is required by predicting special reflection tokens and performing self-critique during generation. Similarly, SeaKR [17] derives uncertainty signals from internal hidden representations and triggers retrieval only when the estimated uncertainty exceeds a learned threshold. While effective, these approaches rely on uncertainty estimates obtained from the internal states of the language model. In contrast, our approach externalizes uncertainty estimation by computing it directly from the similarity-score distribution produced by a dense FAISS retriever, without probing or modifying the language model itself.

B. Entropy and Confidence-Aware Retrieval Strategies

Uncertainty has also been explored as an explicit information-theoretic signal at the retriever level. Entropy-based hybrid retrieval methods use normalized entropy of retrieval scores to dynamically adjust retrieval behavior, such as balancing sparse and dense retrieval components or adapting query weighting [18]. In a complementary direction, the Concept-Based RAG model [19] partitions documents into conceptual segments and leverages entropy to characterize topic dispersion, demonstrating that lower-entropy, more focused retrieval yields higher precision than full-document retrieval [19]. This observation motivates the same underlying principle adopted in our work. When the similarity-score distribution is concentrated, a smaller retrieval depth is often sufficient. Beyond controlling how much information to retrieve, confidence-aware mechanisms aim to assess the reliability of retrieved evidence before generation. Corrective RAG (CRAG) [20] introduces a lightweight retrieval evaluator that categorizes retrieved sets into regimes such as correct, incorrect, or ambiguous, and applies different actions accordingly, including directly using the retrieved content, filtering to extract the most relevant segments, or triggering additional external search. Compared to CRAG, our approach follows a simpler relies solely on retrieval-side signals, avoiding the need for learned evaluators while still enabling uncertainty-aware retrieval control.

C. KV Cache Reuse for Efficient RAG Inference

While adaptive retrieval determines what evidence is retrieved and how much context is supplied, the efficiency of RAG systems is often dominated by the cost of processing retrieved context during inference. In Transformer-based serving, the prompt-processing (prefill) stage computes attention key-value (KV) states for all input tokens before the first output token is generated, making Time-To-First-Token (TTFT) a critical latency metric. RAGCache [9] systematically quantified this behavior, showing that inference latency grows almost linearly with input length due to the cost of prefill computation. To mitigate this inefficiency, RAGCache caches intermediate KV tensors of previously retrieved documents using a hierarchical knowledge-tree structure and reuses them when the same documents reappear in subsequent queries [9]. This approach significantly reduces redundant computation and improves TTFT when retrieval results are repeated. However, RAGCache’s reuse mechanism is order sensitive and relies on exact matches of retrieved document sequences. As a result, KV states computed for one ordered sequence of documents cannot be reused when the same documents appear in a different order, since attention computations depend on token positions and ordering. To overcome this, we introduce an additional Layer-2 Semantic Reuse, where caching occurs at the level of aggregate meaning rather than strict token sequence.

D. Flexible KV Reuse and Serving Frameworks

Several recent systems have explored more flexible forms of KV reuse and memory management. KVLink [21] decouples cached KV tensors from positional embeddings and enables reuse through lightweight positional re-alignment mechanisms, allowing cached representations to be reused when document ordering changes. CacheBlend [22] addresses non-prefix reuse by selectively recomputing a small subset of tokens to repair cross-attention when cached contexts are partially modified or reordered, preserving generation quality while reducing TTFT. At the serving-framework level, SGLang [23] introduces radix-tree based KV cache organization to maximize prefix reuse across concurrent requests, of KV caches to support efficient prefix matching and scheduling, while vLLM’s PagedAttention [24] improves KV memory utilization and throughput by managing attention states in paging-style blocks.

Collectively, these studies demonstrate that improving RAG performance requires effective reuse of intermediate representations and relaxed reuse constraints that can tolerate variations in document ordering and composition, motivating more flexible caching strategies for dynamic retrieval scenarios.

III. METHODS

The overall system architecture is illustrated in Fig. 1 and consists of two phases: offline preparation and online query-time execution. The proposed framework is a system-level simulator designed to study retrieval behavior and prompt-prefill efficiency in RAG systems. Rather than implementing a full LLM serving stack or modifying the underlying language model, the simulator explicitly models retrieval decisions, key-value (KV) cache reuse mechanisms, and their impact on prompt-processing latency, measured as Time-To-First-Token (TTFT). In this setting, KV reuse is abstracted as a reduction in the number of tokens that require prefill computation, allowing controlled analysis of latency gains under realistic query streams.

In the offline phase, the TriviaQA [10] corpus is first cleaned and normalized to obtain a standardized set of question-passage pairs. The resulting passages are encoded into dense vector representations using a Sentence-Transformer embedding model, and indexed with FAISS for efficient nearest-neighbor search [15]. The constructed FAISS index and accompanying metadata (passage texts and identifiers) are persisted and loaded during online evaluation.

In the online phase, an incoming user query is embedded using the same Sentence-Transformer model and used to retrieve the top- $k_{\max}$ candidate passages from the FAISS index. The similarity-score distribution of these candidates is then analyzed to compute a normalized entropy value, which serves as a retriever-side uncertainty signal. Based on this entropy, an adaptive controller selects a query-specific retrieval depth, determining how many retrieved passages are modeled as part of the query context for prefill-cost estimation.

After adaptive context selection, the system applies a two-stage reuse mechanism to reduce prefill computation. First, Layer-1 prefix reuse checks whether the ordered sequence

of retrieved passage identifiers matches a previously cached prefix, enabling exact reuse of cached KV states. If no prefix match is found, Layer-2 semantic reuse compares the current retrieved context against previously observed semantic prototypes and enables partial reuse when sufficient semantic similarity is detected. These mechanisms operate sequentially and are evaluated under both fixed and entropy-guided retrieval policies.

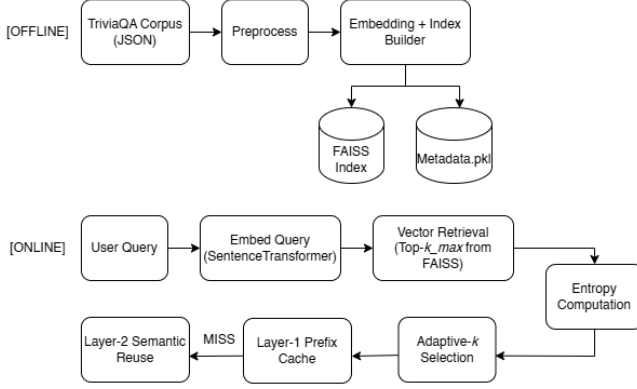


Fig. 1. System architecture of the proposed entropy-guided retrieval and two-layer semantic reuse framework.

A. Data Preprocessing and Index Construction

The experiments are conducted on the TriviaQA question answering corpus [10], which provides a large collection of natural-language questions paired with gold answers and associated candidate evidence passages. Each raw entry contains a question identifier, a question string, an answer field, and a list of retrieved web search results that serve as potential evidence. Since the original format is optimized for dataset storage and annotation rather than retrieval-centric experimentation, the corpus is first transformed into a cleaned, standardized representation and then converted into a dense retrieval index.

In the preprocessing stage, each record is normalized through consistent whitespace cleaning applied to the question text, the answer string, and each evidence field. Candidate evidence passages are extracted from the search results list, where each item typically includes a title, a short description snippet, an optional rank, and a source URL. For retrieval purposes, the title and description are concatenated into a single passage text, forming a compact evidence unit that preserves the most informative content of the search result. Evidence entries containing no usable textual content are discarded. To enforce a uniform retrieval budget and avoid extreme variability across queries, the number of retrieved evidence passages per question is capped at $k_{\max} = 8$. Records are considered valid only if they contain a non-empty identifier, question, answer, and at least one retained passage. After filtering, the cleaned corpus contains 11,260 valid question records out of 11,313 raw records, with 53 records discarded due to missing or unusable fields.

Following cleaning, the dataset is converted into a form suitable for dense retrieval by flattening question-level records into a passage-level corpus. Each retained passage is treated as an independent retrieval unit, while maintaining metadata links to its originating question identifier, its rank in the original search results, and its source URL. Across all questions, this process yields a total of 89,363 passages. Each passage is encoded using the Sentence-Transformer model all-MiniLM-L6-v2 [11], which maps text into a fixed 384-dimensional semantic embedding space. The resulting embeddings are indexed using FAISS [12], configured for inner-product similarity over ℓ_2 -normalized vectors, which corresponds to cosine similarity retrieval in practice.

The final index therefore contains 89,363 dense vectors of dimensionality 384. Alongside the index, a metadata store is maintained to enable mapping retrieved vector identifiers back to their original passage texts and associated question-level information during evaluation. Importantly, the index is used exclusively to simulate retrieval behavior and similarity score distributions. Retrieved passages are not fed into a live language model, but instead serve as inputs to the retrieval uncertainty estimation and prefill-cost simulation described in subsequent sections.

B. Entropy-Guided Adaptive Retrieval

Entropy is a fundamental concept from information theory that quantifies the amount of uncertainty or unpredictability associated with a random variable [25]. In his formulation of the “fundamental problem of communication,” Shannon characterizes communication as the task of correctly identifying which message was generated by a source based on a received signal [26]. Shannon Entropy formalizes this notion as the average information content of a stochastic process, with higher entropy indicating greater uncertainty.

Similarly, in the context of Retrieval-Augmented Generation, entropy provides a natural signal for modeling retriever confidence. Given a user query q , the dense retriever described in the data preprocessing stage returns the top- $k_{\max}$ candidate passages along with their similarity scores. In our system, similarity is computed using cosine similarity between normalized embedding vectors, which measures semantic closeness by the angle between vectors in the embedding space [27]. Let the retrieved similarity scores be denoted as

$$\mathbf{s} = [s_1, s_2, \dots, s_{k_{\max}}] \quad (1)$$

where $k_{\max} = 8$ in all experiments.

To convert this score vector into a probabilistic representation suitable for entropy computation, we apply a temperature-scaled softmax function. The similarity scores are treated as logits and transformed into a categorical probability distribution:

$$p_i = \frac{\exp(s_i/\tau)}{\sum_{j=1}^{k_{\max}} \exp(s_j/\tau)} \quad (2)$$

where $\tau > 0$ is the temperature parameter. The temperature controls the sharpness of the resulting distribution. Smaller

values of τ concentrate probability mass on the highest-scoring passages, indicating higher retriever confidence, while larger values produce a more uniform distribution, reflecting greater uncertainty [28]. In this work, the temperature is fixed to $\tau = 0.1$, which empirically yields a stable separation between confident and uncertain retrieval cases across the dataset.

Giving the resulting probability distribution, retrieval uncertainty is quantified using Shannon entropy:

$$H(\mathbf{s}) = - \sum_{i=1}^{k_{\max}} p_i \log p_i \quad (3)$$

Since entropy increases with the number of retrieved candidates, it is normalized to the interval $[0, 1]$ by dividing by the maximum possible entropy $\log(k_{\max})$:

$$H_{\text{norm}}(\mathbf{s}) = \frac{H(\mathbf{s})}{\log(k_{\max})} \quad (4)$$

Intuitively, a low value of H_{norm} indicates that the similarity distribution is highly concentrated, meaning that one or a small number of passages dominate the retrieval results and the retriever is confident. Conversely, a high H_{norm} corresponds to a diffuse distribution in which many passages receive comparable similarity scores, reflecting higher uncertainty. Importantly, this uncertainty estimate is computed solely from the retriever’s output scores and does not require access to, or modification of, the internal representations of the language model.

In the proposed framework, the normalized entropy signal is used exclusively to control the number of retrieved passages considered in the simulation. It determines a query-specific retrieval depth for modeling downstream prefill cost and reuse behavior.

C. Evaluation Protocol for Adaptive Retrieval

Rather than always using the maximum retrieval depth $k_{\max}$, the system selects a smaller, query-specific retrieval depth based on the normalized entropy:

$$k = f(H_{\text{norm}}) \quad (5)$$

where $f(\cdot)$ is a piecewise-constant mapping defined by entropy thresholds. Specifically, given thresholds (t_1, t_2, t_3) and candidate retrieval depths (k_1, k_2, k_3, k_4) , the adaptive controller is defined as

$$k = \begin{cases} k_1, & H_{\text{norm}} \leq t_1, \\ k_2, & t_1 < H_{\text{norm}} \leq t_2, \\ k_3, & t_2 < H_{\text{norm}} \leq t_3, \\ k_4, & H_{\text{norm}} > t_3. \end{cases} \quad (6)$$

In our implementation, $k_{\max} = 8$ and the softmax temperature is fixed to $\tau = 0.1$. The entropy thresholds are calibrated empirically using the full cleaned TriviaQA dataset consisting of 11,260 queries. For each query, we retrieve the top- $k_{\max}$ passages, compute H_{norm} , and determine the smallest retrieval depth sufficient to recover evidence containing the gold answer string. Formally, letting $\mathcal{D}_{1:k}$ denote the first k

retrieved passages ranked by similarity, we define the required retrieval depth as

$$k_{\text{needed}} = \min \{k \in \{1, \dots, k_{\max}\} : \text{Answer} \subseteq \mathcal{D}_{1:k}\} \quad (7)$$

The containment check is implemented via a case-insensitive substring match against passage text. While this criterion does not guarantee correct generation, it provides a scalable and consistent proxy for retrieval sufficiency during calibration.

Across the dataset, the normalized entropy distribution has mean 0.907 and standard deviation 0.084, while the average required retrieval depth is $k_{\text{needed}} = 4.54$. The 25th, 50th, and 75th percentiles of H_{norm} are 0.880, 0.929, and 0.963, respectively. These values are used as (t_1, t_2, t_3) in Eq. (6). Partitioning the dataset using these thresholds yields four approximately equal-sized buckets of 2,815 queries each, with the following statistics:

- $H_{\text{norm}} \leq 0.880$: mean $k_{\text{needed}} = 4.56$
- $0.880 < H_{\text{norm}} \leq 0.929$: mean $k_{\text{needed}} = 4.38$
- $0.929 < H_{\text{norm}} \leq 0.963$: mean $k_{\text{needed}} = 4.30$
- $H_{\text{norm}} > 0.963$: mean $k_{\text{needed}} = 4.92$

The relatively high entropy observed across TriviaQA queries can be attributed to the dataset’s construction. TriviaQA questions are open-domain, short, and often underspecified, leading to multiple passages with comparable semantic relevance. As a result, the similarity-score distribution produced by dense retrieval is often diffuse rather than sharply peaked, yielding higher entropy values even when the correct evidence is present. This characteristic makes it a particularly suitable benchmark for evaluating uncertainty-aware and adaptive retrieval strategies.

To quantify the impact of adaptive retrieval on context size and evidence coverage, we compare fixed- k retrieval against entropy-guided adaptive retrieval on the same set of queries. Two primary metrics are reported. First, the retrieved token budget is estimated by whitespace tokenization of each retrieved passage and summing across the retrieval set, providing a consistent approximation of content length. Second, we report an accuracy proxy defined as the proportion of queries for which the gold answer string appears in the retrieved passages, consistent with Eq. (7).

We evaluate three adaptive retrieval strategies corresponding to different (k_1, k_2, k_3, k_4) configurations, summarized in Table I. These strategies trade off retrieval efficiency against evidence coverage.

TABLE I
COMPARISON OF ADAPTIVE RETRIEVAL STRATEGIES UNDER
ENTROPY-GUIDED CONTROL

Strategy	k Values	Token Reduction	Accuracy Drop
Aggressive	(3,5,7,8)	28.5%	4.25%
Balanced	(4,5,7,8)	25.3%	3.30%
Conservative	(5,6,7,8)	19.0%	2.00%

Based on this comparison, we adopt the Balanced configuration $(k_1, k_2, k_3, k_4) = (4, 5, 7, 8)$ in all subsequent experiments. This setting provides a strong reduction in retrieved

context while maintaining a favorable accuracy–efficiency trade-off. Under this configuration, the adaptive retrieval rule in Eq. (6) instantiates to

$$k = \begin{cases} 4, & H_{\text{norm}} \leq 0.880, \\ 5, & 0.880 < H_{\text{norm}} \leq 0.929, \\ 7, & 0.929 < H_{\text{norm}} \leq 0.963, \\ 8, & H_{\text{norm}} > 0.963. \end{cases} \quad (8)$$

This rule is applied uniformly across all online experiments and governs the number of retrieved passages considered by the simulator for each query.

D. Layer-1 Prefix Reuse (RAGCache Simulation)

Layer-1 prefix reuse simulates the core idea introduced in RAGCache [9], which targets the dominant latency bottleneck in Retrieval-Augmented Generation: redundant prompt-prefill computation during inference. In standard RAG pipelines, retrieved passages are appended to the input sequence supplied to the language model. When the same documents are retrieved repeatedly across queries, the model recomputes identical KV states during the prefill phase, leading to increased Time-To-First-Token (TTFT) [29].

In our work, however, we do not execute a real language model or manipulate its internal KV tensors. Instead, we simulate the behavior of prefix-based KV reuse by modeling how many tokens would require prefill computation under different cache reuse scenarios. TTFT is therefore treated as proportional to the number of tokens whose KV states must be recomputed.

The key observation behind prefix reuse is that Transformer attention is deterministic given an identical token sequence and ordering. Consequently, if a query retrieves the same documents in the same order as a previous query, the KV states corresponding to those documents can, in principle, be reused without recomputation. Layer-1 reuse captures this behavior by caching exact prefixes of retrieved document sequences and reusing them when an identical prefix appears.

Formally, let a query retrieve an ordered list of document identifiers

$$\mathbf{d} = [d_1, d_2, \dots, d_k] \quad (9)$$

where each document d_i contributes t_i tokens to the augmented input sequence. Let t_q denote the number of query (instruction) tokens. The total number of tokens processed during prompt prefill is

$$T_{\text{total}} = t_q + \sum_{i=1}^k t_i \quad (10)$$

Layer-1 maintains a prefix cache implemented as a knowledge tree. Each node in the tree corresponds to a cached document prefix $(d_1, \dots, d_j)$ and stores the cumulative number of tokens contributed by that prefix. Given a new retrieval sequence $\mathbf{d}$, the system computes the length of the longest cached prefix

$$\ell = \max \{j \leq k : (d_1, \dots, d_j) \in \mathcal{C}\} \quad (11)$$

where $\mathcal{C}$ denotes the set of cached prefixes.

If $\ell > 0$, the KV computation corresponding to the first ℓ documents is assumed to be reusable. The number of reusable tokens is

$$T_{\text{cached}} = \sum_{i=1}^{\ell} t_i \quad (12)$$

Only the remaining tokens require simulated prefill computation.

To model prefill latency, we approximate TTFT as proportional to the number of tokens whose KV states must be computed. Specifically, we assume a constant per-token prefill cost c , yielding

$$\text{TTFT}_{\text{Base}} = c \cdot T_{\text{total}} \quad (13)$$

$$\text{TTFT}_{\text{Layer-1}} = c \cdot (T_{\text{total}} - T_{\text{cached}}) \quad (14)$$

In all experiments, the per-token cost is fixed to $c = 0.05$ milliseconds. This value does not aim to reproduce hardware-specific latency, but rather, it provides a consistent scaling factor that enables relative TTFT reductions to be compared across retrieval and reuse strategies. Because the same constant is applied in all cases, observed TTFT improvements directly reflect reductions in recomputed token volume.

After processing a query, the retrieved document sequence is inserted into the knowledge tree. All prefixes (d_1) , (d_1, d_2) , $\dots$, $(d_1, \dots, d_k)$ are added if they do not already exist. Each cache node records both the cumulative token count of its prefix and a last-access timestamp, enabling efficient reuse tracking and supporting potential cache management policies.

Algorithm 1 summarizes the Layer-1 prefix reuse procedure.

Algorithm 1 Layer-1 Prefix Reuse (RAGCache Simulation)

```

1:  $T_{\text{total}} \leftarrow t_q + \sum_{i=1}^k t_i$ 
2:  $\ell \leftarrow 0$ 
3: for  $j = k$  down to 1 do
4:   if  $(d_1, \dots, d_j)$  exists in prefix cache then
5:      $\ell \leftarrow j$ 
6:   break
7: end if
8: end for
9:  $T_{\text{cached}} \leftarrow \sum_{i=1}^{\ell} t_i$ 
10:  $\text{TTFT}_{\text{Base}} \leftarrow c \cdot T_{\text{total}}$ 
11:  $\text{TTFT}_{\text{Layer-1}} \leftarrow c \cdot (T_{\text{total}} - T_{\text{cached}})$ 
12: for  $j = 1$  to  $k$  do
13:   Insert prefix  $(d_1, \dots, d_j)$  into cache if absent
14: end for
15: return  $\text{TTFT}_{\text{Base}}, \text{TTFT}_{\text{Layer-1}}$ 

```

E. Layer-2 Semantic Reuse (Approximate KV Reuse)

Layer-2 semantic reuse extends caching beyond exact-prefix matching by enabling approximate KV reuse when the retrieved context is not identical but remains semantically close to previously observed contexts. This design is motivated by realistic query streams, where small changes in query phrasing or retrieval ranking often alter the order or composition of

retrieved passages. Such variations frequently invalidate Layer-1 exact-prefix reuse, even though the underlying topic, intent, and supporting evidence remain largely unchanged.

The key idea behind Layer-2 is that semantic similarity implies partial computational overlap. Because exact reuse of attention key-value (KV) states requires identical token sequences, it is not mathematically valid to reuse KV states across arbitrarily different token sequences. However, semantically related retrieval contexts often share substantial structural commonality, such as similar narrative framing, overlapping factual explanations, or repeated background explanations. For instance, a query about “apple” and another about “red fruit” cannot reuse their full KV cache, but they may share a common conceptual frame that can be partially reused.

In this work, Layer-2 is a system-level simulation that estimates how much prompt-prefill computation could be avoided if semantically retrieved contexts allowed partial reuse of previously computed intermediate representations. It adopts a conservative approximation: when two retrieved contexts are sufficiently similar, only a fixed fraction of the document-prefill computation is assumed to be reusable, while the remaining portion is recomputed to account for semantic drift.

Let the retrieved context after adaptive- k selection be the ordered list of passage texts

$$\mathcal{X} = [x_1, x_2, \dots, x_k] \quad (15)$$

Each passage x_i is embedded using the same Sentence-Transformer model employed during index construction. Let $\mathbf{e}_i \in \mathbb{R}^d$ denote the ℓ_2 -normalized embedding of passage x_i , such that cosine similarity is computed as a dot product. To obtain a single representation for the entire retrieved context, Layer-2 applies mean pooling over the passage embeddings:

$$\mathbf{v}(\mathcal{X}) = \frac{1}{k} \sum_{i=1}^k \mathbf{e}_i \quad (16)$$

This vector is then renormalized to unit length

$$\hat{\mathbf{v}}(\mathcal{X}) = \frac{\mathbf{v}(\mathcal{X})}{\|\mathbf{v}(\mathcal{X})\|_2} \quad (17)$$

yielding a compact semantic signature of the retrieved document set. Unlike Layer-1, which keys reuse on exact document identifiers and ordering, Layer-2 operates on the aggregate semantic meaning of the retrieved context.

Layer-2 maintains a growing set of previously observed semantic prototypes

$$\mathcal{P} = \{\hat{\mathbf{p}}_1, \hat{\mathbf{p}}_2, \dots, \hat{\mathbf{p}}_m\} \quad (18)$$

where each prototype corresponds to the pooled semantic representation of a past retrieved context. For a new query, semantic similarity is computed as the maximum cosine similarity between the current context vector and all stored prototypes:

$$s_{\max} = \max_{1 \leq j \leq m} \hat{\mathbf{v}}(\mathcal{X})^\top \hat{\mathbf{p}}_j \quad (19)$$

If no prototype exists ($m = 0$), semantic reuse is not possible and the query is rejected by default.

A reuse decision is made by comparing $s_{\max}$ against a fixed similarity threshold θ . In our implementation, θ is set to 0.90, reflecting a high semantic overlap requirement that prevents reuse across loosely related contexts. The decision rule is

$$\text{reuse} = \begin{cases} 1, & s_{\max} \geq \theta, \\ 0, & s_{\max} < \theta. \end{cases} \quad (20)$$

When semantic reuse is accepted, Layer-2 assumes that a fixed fraction of the document-prefill computation can be skipped. Let t_i denote the number of tokens contributed by passage x_i , and define the total number of document tokens as

$$T_{\text{doc}} = \sum_{i=1}^k t_i \quad (21)$$

The number of tokens whose prefill computation is skipped is modeled as

$$T_{\text{skip}} = \lfloor \rho \cdot T_{\text{doc}} \rfloor \quad (22)$$

where ρ is the reuse ratio. In this work, ρ is fixed to 0.6, meaning that at most 60% of the document-prefill computation is assumed to be reusable. This value is intentionally chosen below 1.0 to reflect that semantic similarity does not imply perfect equivalence. Some recomputation is always required to adapt the cached representations to the new context.

Importantly, this does not mean that 60% of the retrieved documents or their tokens are removed from the model input. All retrieved passages remain part of the simulated input. The reuse ratio only affects how much prefill computation is assumed to be skipped in the latency model.

Using the same per-token prefill cost $c = 0.05$ ms as in the Layer-1 simulation, the effective Time-To-First-Token under Layer-2 reuse is modeled as

$$\text{TTFT}_{\text{Layer-2}} = c \cdot (T_{\text{total}} - T_{\text{cached}} - T_{\text{skip}}) \quad (23)$$

where T_{total} is the total number of tokens in the simulated input (query plus retrieved documents), T_{cached} denotes the number of tokens reused via Layer-1 exact-prefix matching (if any), and T_{skip} is defined in Eq. (22). This formulation makes Layer-2 strictly additive to Layer-1. Semantic reuse is only applied to the portion of computation not already covered by exact-prefix reuse.

After each query is processed, the semantic prototype $\hat{\mathbf{v}}(\mathcal{X})$ of the retrieved context is added to the prototype set $\mathcal{P}$. Over time, this allows Layer-2 to accumulate a diverse set of semantic anchors corresponding to previously observed retrieval patterns. Unlike Layer-1, which stores all document prefixes explicitly, Layer-2 stores only one vector per query, resulting in a compact memory footprint. Acceptance and rejection statistics are tracked to quantify how often semantic reuse is triggered and to measure its contribution to TTFT reduction.

It is important to emphasize that Layer-2 does not claim to faithfully reproduce exact Transformer attention dynamics. Instead, it serves as a controlled abstraction that estimates the potential benefits of semantic reuse under realistic conditions

where retrieval overlap is conceptual rather than syntactic. By combining conservative similarity thresholds ($\theta = 0.90$) with partial reuse ($\rho = 0.60$), the simulator prioritizes safety, interpretability, and reproducibility over aggressive acceleration.

Algorithm 2 summarizes the Layer-2 semantic reuse procedure.

Algorithm 2 Layer-2 Semantic Reuse

```

1: Compute passage embeddings  $\mathbf{e}_i$  for each  $x_i$ 
2: Compute pooled context vector  $\hat{\mathbf{v}}(\mathcal{X})$  using (17)
3: if  $\mathcal{P}$  is empty then
4:   reject semantic reuse
5:    $T_{\text{skip}} \leftarrow 0$ 
6: else
7:    $s_{\text{max}} \leftarrow \max_{\hat{\mathbf{p}} \in \mathcal{P}} \hat{\mathbf{v}}(\mathcal{X})^\top \hat{\mathbf{p}}$ 
8:   if  $s_{\text{max}} \geq \theta$  then
9:     accept semantic reuse
10:     $T_{\text{skip}} \leftarrow \lfloor \rho \cdot \sum_{i=1}^k t_i \rfloor$ 
11:   else
12:     reject semantic reuse
13:     $T_{\text{skip}} \leftarrow 0$ 
14:   end if
15: end if
16: Update prototype set:  $\mathcal{P} \leftarrow \mathcal{P} \cup \{\hat{\mathbf{v}}(\mathcal{X})\}$ 
17: return  $T_{\text{skip}}$ 

```

F. Interactive Query and Online Cache Evolution

To evaluate the behavior of the proposed system under realistic, sequential usage, we implement an interactive query pipeline that simulates an online Retrieval-Augmented Generation (RAG) serving scenario. The system processes incoming queries one at a time, while the cache state evolves incrementally across queries. This setup allows us to observe how entropy-guided retrieval and two-layer reuse behave under realistic query streams rather than isolated, stateless evaluations.

Given a user query q , the system first computes its dense embedding using the same Sentence-Transformer model employed during index construction. The FAISS [12] index is then queried to retrieve the top- $k_{\text{max}} = 8$ candidate passages together with their similarity scores. Retrieval uncertainty is estimated by computing the normalized entropy H_{norm} of the similarity-score distribution using temperature $\tau = 0.1$, as described in Section III-C. When entropy-aware retrieval is enabled, the adaptive controller selects a query-specific retrieval depth $k = f(H_{\text{norm}})$. Otherwise, a fixed retrieval depth k_{max} is used. This dual-mode evaluation enables a direct comparison between entropy-guided and static retrieval policies under identical retrieval conditions.

After retrieval, the selected passages are passed to the caching subsystem, where reuse decisions are applied in staged manner. First, Layer-1 exact-prefix reuse is evaluated. The system checks whether the ordered sequence of retrieved document identifiers matches a previously cached prefix in the knowledge tree. If a prefix hit occurs, the corresponding portion of the simulated prefill computation is reused, and only the remaining tokens are recomputed. If no prefix reuse is possible, the system proceeds to Layer-2 semantic reuse. In Layer-2, the retrieved document set is encoded into a pooled

semantic context vector and compared against previously stored semantic prototypes. If the maximum cosine similarity exceeds the threshold $\theta = 0.9$, semantic reuse is accepted and a fraction $\rho = 0.6$ of the document-prefill computation is assumed to be reusable.

Prefill latency is modeled as proportional to the number of tokens requiring KV computation, using a constant per-token cost $c = 0.05$ ms. For each query, the system reports the baseline TTFT (no reuse), the effective TTFT after applying reuse, and the relative TTFT gain. In addition to latency metrics, an accuracy proxy is computed to monitor evidence coverage. Specifically, the system checks whether the retrieved passages contain the gold answer string using fuzzy string matching [30] with a similarity threshold of 80. While this proxy does not measure generation correctness, it provides a consistent and scalable signal indicating whether retrieval remains sufficient under different retrieval depths and reuse configurations.

To emulate a warm-start deployment scenario, the system performs a short bootstrap phase in which the first ten queries from dataset are processed sequentially and inserted into both the Layer-1 prefix cache and the Layer-2 semantic prototype set. These initial queries establish reusable prefixes and semantic anchors that subsequent interactive queries may benefit from. Table II lists the ten queries used during this cache warmup phase together with their corresponding gold answers.

TABLE II
INITIAL BOOTSTRAP QUERIES USED TO WARM LAYER-1 AND LAYER-2
CACHES BEFORE INTERACTIVE EVALUATION

Question	Answer
1. Who was the man behind The Chipmunks?	David Seville
2. What star sign is Jamie Lee Curtis?	Scorpio
3. Which Lloyd Webber musical premiered in the US on 10th December 1993?	Sunset Boulevard
4. Who was the next British Prime Minister after Arthur Balfour?	Campbell-Bannerman
5. Who had a 70s No 1 hit with Kiss You All Over?	Exile
6. What claimed the life of singer Kathleen Ferrier?	Cancer
7. Rita Coolidge sang the title song for which Bond film?	Octopussy
8. To the nearest million, what is the population of Australia?	18 million
9. What was the last US state to reintroduce alcohol after prohibition?	Utah
10. Which actress was voted Miss Greenwich Village in 1942?	Lauren Bacall

During interactive operation, each processed query incrementally updates the cache state. The ordered sequence of retrieved document identifiers is inserted into the Layer-1 knowledge tree, enabling exact-prefix reuse for future queries with overlapping retrieval results. In parallel, the pooled semantic context vector of the retrieved passages is added to the Layer-2 prototype set, expanding the set of semantic anchors available for approximate reuse.

This online update mechanism allows the simulator to model cache evolution under realistic query streams, where reuse opportunities emerge gradually as related questions are encountered over time. Importantly, this adaptation occurs entirely at query time and does not require retraining the retriever, rebuilding the index, or performing offline recomputation.

IV. RESULTS

The TriviaQA [10] dataset primarily consists of distinct, independently phrased questions. To reflect realistic interactive usage, where users may repeat, rephrase, or ask related questions over time, we evaluate the system under multiple online query interaction patterns that exercise different reuse paths and uncertainty regimes.

We consider four representative interaction patterns: (i) exact repetition of previously observed questions (Layer-1 prefix reuse), (ii) semantically similar but lexically different questions (Layer-2 semantic reuse), (iii) unrelated questions (no reuse), and (iv) back-to-back queries where an initial semantic reuse enables prefix reuse on a subsequent query.

All scenarios are evaluated under two retrieval policies: Entropy OFF, which always retrieves a fixed $k_{\max} = 8$ passages, and Entropy ON, which applies entropy-guided adaptive retrieval as described in Section III. For each query, we report the reuse path (NONE, L1, or L2), the selected retrieval depth k , relative TTFT gain, and Layer-2 metadata when applicable.

A. Pattern 1: Exact Question Repetition

Table III reports results for queries that exactly match previously observed questions.

TABLE III
PATTERN 1: EXACT QUESTION REPETITION (LAYER-1 REUSE)

Query	Entropy	Path	k	TTFT Gain
Who was the man behind The Chipmunks?	OFF	L1	8	89.71%
	ON	L1	8	89.71%
Rita Coolidge sang the title song for which Bond film?	OFF	L1	8	89.97%
	ON	L1	4	82.02%

In both examples, the retrieved document sequence and ordering exactly match with cached prefixes of questions one and seven in Table II, enabling full reuse of the corresponding KV states. As a result, nearly the entire document-prefix stage is skipped, yielding TTFT reductions close to 90%. For the Chipmunks query, entropy-based adaptation selects the same retrieval depth ($k = 8$), producing identical TTFT gains under both policies. In contrast, for the Bond film query, Entropy ON confidently reduces the retrieval depth to $k = 4$, slightly lowering the TTFT gain but substantially reducing the retrieved context size.

TABLE IV
PATTERN 2: SEMANTIC SIMILARITY TRIGGERING LAYER-2 REUSE

Query	Entropy	Path	k	TTFT Gain	L2 Sim
Which person created the Chipmunks?	OFF	L2	8	54.04%	0.969
	ON	L2	8	54.04%	0.969
Which Bond film features 'All Time High' by Rita Coolidge?	OFF	L2	8	53.99%	0.978
	ON	L2	7	53.29%	0.947

B. Pattern 2: Semantically Similar Queries

Table IV presents results for semantically similar but lexically different queries.

In these cases, minor rephrasings change retrieval rankings or document ordering, preventing exact-prefix reuse and causing Layer-1 to miss. However, the high cosine similarity scores (0.947–0.978) between the current retrieved contexts and previously cached contexts trigger Layer-2 semantic reuse. Although Layer-2 recomputes part of the document-prefix stage, it still skips a substantial fraction of KV computation, resulting in consistent TTFT reductions exceeding 50%. Under Entropy ON, the retrieval depth for one query ($k = 7$ instead of 8), marginally lowering TTFT gain while again decreasing token usage. These results highlight the ability of Layer-2 to recover reuse opportunities when exact matching fails.

C. Pattern 3: Unrelated Queries

Table V shows results for queries that are unrelated to previously observed contexts.

TABLE V
PATTERN 3: UNRELATED QUERIES (NO REUSE)

Query	Entropy	Path	k	TTFT Gain	L2 Sim
What is the capital city of Turkiye?	OFF	NONE	8	0.00%	0.289
	ON	NONE	8	0.00%	0.298
What color is the apple?	OFF	NONE	8	0.00%	0.071
	ON	NONE	8	0.00%	0.064

In these cases, both Layer-1 and Layer-2 reuse correctly fail to activate due to absence of semantic overlap, as reflected by similarity scores. Consequently, the system falls back to full document-prefix computation, yielding zero TTFT gain under both Entropy OFF and Entropy ON settings. This behavior demonstrates that the reuse mechanisms are conservative and do not introduce unsafe approximations when semantic similarity is insufficient.

D. Pattern 4: Back-to-Back Queries

Table VI illustrates a back-to-back query scenario that highlights cache evolution over time.

The first query triggers Layer-2 semantic reuse, achieving approximately 50% TTFT reduction despite the absence of exact-prefix matches. Crucially, after processing this query, the retrieved document sequence is inserted into the Layer-1 prefix cache. When the same query is issued again, exact-prefix reuse becomes possible, enabling Layer-1 reuse and producing

TABLE VI
PATTERN 4: BACK-TO-BACK QUERIES ENABLING LAYER-1 REUSE

Query	Entropy	Path	k	TTFT Gain
Australia population rounded? (first)	OFF	L2	8	53.17%
	ON	L2	5	50.00%
Australia population rounded? (repeat)	OFF	L1	8	88.73%
	ON	L1	5	83.51%

TTFT gains close to 90%. This transition demonstrates how Layer-2 semantic reuse can bootstrap Layer-1 prefix reuse in subsequent interactions, effectively converting approximate semantic overlap into exact reuse over time. Entropy ON further reduces retrieval depth for both queries, slightly lowering TTFT gains but consistently reducing context size.

V. DISCUSSION & CONCLUSION

A. Main Findings

This work examines how uncertainty-aware retrieval and multi-level reuse can reduce the dominant inference-time cost of Retrieval-Augmented Generation (RAG), namely Time-To-First-Token (TTFT). The results show that entropy-guided adaptive retrieval consistently reduces retrieved context size, while KV reuse mechanisms significantly reduce prefill computation when reuse opportunities arise. Importantly, these two mechanisms are complementary. Adaptive retrieval controls how much context is retrieved, while reuse mechanisms reduce the cost of processing that context.

B. Insights Beyond Prior Work

The experiments reveal exact-prefix reuse, as in RAGCache [9], is highly effective but fragile under realistic queries. A key insight from this study is that retrieval-side uncertainty can be estimated without modifying the language model, yet still provide meaningful control over retrieval depth. Another important contribution is demonstrating that semantic similarity at the context level can be used as a proxy for computational overlap. While prior systems rely on exact token sequences, Layer-2 reuse shows that conservative, approximate reuse can recover a large fraction of the benefits of exact reuse under realistic query streams. Moreover, semantic reuse can bootstrap exact-prefix reuse over time, converting approximate overlap into exact reuse in subsequent interactions.

C. Effectiveness and Limitations

Layer-1 prefix reuse consistently achieves the largest TTFT reductions (close to 90%) when applicable, but provides no benefit when retrieval order or composition changes. Layer-2 reuse is less aggressive but significantly more robust, delivering consistent TTFT reductions of around 50% for semantically related queries. Entropy-guided retrieval reduces token usage across all scenarios, though its interaction with reuse introduces a trade-off between absolute TTFT gain and overall computational efficiency.

The main limitation of this work is its system-level simulation design. KV reuse is modeled as skipped prefill computation rather than implemented within an actual Transformer inference stack. Additionally, Layer-2 reuse relies on a fixed reuse ratio and a single similarity threshold, which may not optimally capture reuse potential across diverse contexts.

D. Future Directions

Future work includes integrating the proposed mechanisms into a real LLM serving system to validate the simulator’s assumptions under true attention computation. More adaptive or learned reuse policies could replace fixed thresholds and reuse ratios. Extending evaluation to multi-turn conversations and end-to-end generation quality would further clarify the practical impact of semantic reuse in deployed RAG systems.

E. Conclusion

Overall, this study shows that combining entropy-guided adaptive retrieval with two-level reuse offers a practical approach to improving RAG efficiency. By leveraging retrieval uncertainty and semantic overlap, the proposed framework extends KV reuse beyond exact matching and demonstrates meaningful TTFT reductions under realistic interactive workloads, without modifying the underlying language model.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *arXiv preprint, arXiv:1706.03762*, Jun. 2017, doi: 10.48550/arXiv.1706.03762.
- [2] Y. Kim, M. Choi, S. Cho, H. J. Kim, S.-g. Lee, and T. Kim, “Reliability across parametric and external knowledge: Understanding knowledge handling in LLMs,” *arXiv preprint, arXiv:2502.13648v1*, Feb. 2025, licensed under CC BY 4.0.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, arXiv:2005.11401, doi: 10.48550/arXiv.2005.11401.
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint, arXiv:2312.10997*, Dec. 2023, doi: 10.48550/arXiv.2312.10997.
- [5] Q. Zhang, S. Chen, Y. Bei, Z. Yuan, H. Zhou, Z. Hong, H. Chen, Y. Xiao, C. Zhou, J. Dong, Y. Chang, and X. Huang, “A survey of graph retrieval-augmented generation for customized large language models,” *arXiv preprint, arXiv:2501.13958*, Jan. 2025, doi: 10.48550/arXiv.2501.13958.
- [6] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, arXiv:2401.09670, doi: 10.48550/arXiv.2401.09670.
- [7] M. Horton, Q. Cao, C. Sun, Y. Jin, S. Mehta, M. Rastegari, and M. Nabi, “KV prediction for improved time to first token,” *arXiv preprint, arXiv:2410.08391*, Oct. 2024, doi: 10.48550/arXiv.2410.08391.
- [8] Y. Zhang, Y. Liu, H. Yuan, Z. Qin, Y. Yuan, Q. Gu, and A. C.-C. Yao, “Tensor product attention is all you need,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2501.06425, doi: 10.48550/arXiv.2501.06425.
- [9] C. Jin, Z. Zhang, X. Jiang, F. Liu, X. Liu, X. Liu, and X. Jin, “RAGCache: Efficient knowledge caching for retrieval-augmented generation,” *arXiv preprint, arXiv:2404.12457*, Apr. 2024, doi: 10.48550/arXiv.2404.12457.
- [10] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension,” *arXiv preprint, arXiv:1705.03551*, May 2017, doi: 10.48550/arXiv.1705.03551.

- [11] Sentence-Transformers, “all-MiniLM-L6-v2,” Hugging Face model repository. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. [Accessed: Dec. 2025].
- [12] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The Faiss library,” *arXiv preprint*, arXiv:2401.08281, Jan. 2024, doi: 10.48550/arXiv.2401.08281.
- [13] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, arXiv:2004.04906, doi: 10.48550/arXiv.2004.04906.
- [14] D. Xu, Y. Wen, P. Jia, Y. Zhang, W. Zhang, Y. Wang, H. Guo, R. Tang, X. Zhao, E. Chen, and T. Xu, “From single to multi-granularity: Toward long-term memory association and selection of conversational agents,” *arXiv preprint*, arXiv:2505.19549v2, Sep. 2025.
- [15] A. Andoni, P. Indyk, and I. Razenshteyn, “Approximate nearest neighbor search in high dimensions,” *arXiv preprint*, arXiv:1806.09823, Jun. 2018, doi: 10.48550/arXiv.1806.09823.
- [16] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint*, arXiv:2310.11511, Oct. 2023, doi: 10.48550/arXiv.2310.11511.
- [17] Z. Yao, W. Qi, L. Pan, S. Cao, L. Hu, W. Liu, L. Hou, and J. Li, “SeaKR: Self-aware knowledge retrieval for adaptive retrieval-augmented generation,” *arXiv preprint*, arXiv:2406.19215, Jun. 2024, doi: 10.48550/arXiv.2406.19215.
- [18] Anonymous, “Entropy-based dynamic hybrid retrieval for adaptive query weighting in RAG pipelines,” *ACL Rolling Review (ARR) preprint*, May 2025, revised Jul. 2025, licensed under CC BY 4.0.
- [19] C.-Y. Lin and J.-S. R. Jang, “Concept-based RAG models: A high-accuracy fact retrieval approach,” in *Proc. Joint Workshop of the 9th FinNLP, the 6th FNP, and the 1st LLMFinLegal*, Jan. 19–20, 2025, pp. 96–100.
- [20] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv preprint*, arXiv:2401.15884, Jan. 2024, doi: 10.48550/arXiv.2401.15884.
- [21] J. Yang, B. Hou, W. Wei, Y. Bao, and S. Chang, “KVLink: Accelerating large language models via efficient KV cache reuse,” *arXiv preprint*, arXiv:2502.16002, Feb. 2025, doi: 10.48550/arXiv.2502.16002.
- [22] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, “CacheBlend: Fast large language model serving for RAG with cached knowledge fusion,” *arXiv preprint*, arXiv:2405.16444, May 2024, doi: 10.48550/arXiv.2405.16444.
- [23] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” *arXiv preprint*, arXiv:2312.07104, Dec. 2023, doi: 10.48550/arXiv.2312.07104.
- [24] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. ACM Symposium on Operating Systems Principles (SOSP)*, 2023, arXiv:2309.06180, doi: 10.48550/arXiv.2309.06180.
- [25] S. Vajapeyam, “Understanding Shannon’s entropy metric for information,” *arXiv preprint*, arXiv:1405.2061, May 2014, doi: 10.48550/arXiv.1405.2061.
- [26] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [27] P. Xia, L. Zhang, and F. Li, “Learning similarity with cosine similarity ensemble,” *Information Sciences*, vol. 307, pp. 39–52, 2015, doi: 10.1016/j.ins.2015.02.024.
- [28] M. Peeperkorn, T. Kouwenhoven, D. Brown, and A. Jordanous, “Is temperature the creativity parameter of large language models?” *arXiv preprint*, arXiv:2405.00492, May 2024, doi: 10.48550/arXiv.2405.00492.
- [29] E. Sun, Y. Wang, and T. Lan, “Block-attention for efficient RAG,” *arXiv preprint*, arXiv:2409.15355v4, Oct. 2024, licensed under CC BY 4.0.
- [30] W.-Y. Wu, “A method for fuzzy string matching,” in *Proc. International Computer Symposium (ICS)*, Chiayi, Taiwan, 2016, pp. 380–383, doi: 10.1109/ICS.2016.0083.

APPENDICES

Appendix A - Sourcecode

```
1 # src/preprocess.py
2 """
3 Output File: data/processed/triviaqa_clean.jsonl
4 """
5 import json
6 import os
7 from tqdm import tqdm
8
9 # Paths
10 RAW_PATH = "data/raw/triviaqa_raw.json"
11 OUT_PATH = "data/processed/triviaqa_clean.jsonl"
12
13 MAX_DOCS_PER_Q = 8
14
15 def load_raw():
16     if not os.path.exists(RAW_PATH):
17         raise FileNotFoundError(f"Input file not found at: {RAW_PATH}")
18
19     print(f"Loading raw data from {RAW_PATH}...")
20     with open(RAW_PATH, "r", encoding="utf-8") as f:
21         data = json.load(f)
22     # The TriviaQA JSON structure has a "Data" key holding the list
23     return data["Data"]
24
25 def clean_text(text):
26     if not text:
27         return ""
28     return " ".join(text.split())
29
30 def flatten_record(item):
31     """
32     Extracts relevant fields and formats documents.
33     Returns None if the record is invalid.
34     """
35     # 1. Extract Question
36     q = clean_text(item.get("Question", ""))
37
38     # 2. Extract Answer
39     answer_block = item.get("Answer") or {}
40     ans = clean_text(answer_block.get("Value", ""))
41
42     # 3. Extract Documents (SearchResults)
43     raw_search_results = item.get("SearchResults") or []
44
45     docs = []
46     for sr in raw_search_results:
47         title = clean_text(sr.get("Title", ""))
48         desc = clean_text(sr.get("Description", ""))
49         url = sr.get("Url", "")
50
51         if not title and not desc:
52             continue
53
54         merged = f"{title}. {desc}"
55         docs.append({
56             "rank": sr.get("Rank", None),
57             "title": title,
58             "text": merged,
59             "url": url
60         })
61
62     docs = docs[:MAX_DOCS_PER_Q]
63     qid = clean_text(item.get("QuestionId", ""))
64
65     if not qid or not q or not ans or not docs:
66         return None
67
68     return {
69         "id": qid,
70         "question": q,
```

```

71         "answer": ans,
72         "docs": docs # List of dicts
73     }
74
75 def process_all():
76     os.makedirs(os.path.dirname(OUT_PATH), exist_ok=True)
77
78     data = load_raw()
79     total = len(data)
80     saved_count = 0
81
82     print(f"Found {total} records. Processing...")
83
84     with open(OUT_PATH, "w", encoding="utf-8") as out:
85         for item in tqdm(data, desc="Cleaning Data"):
86             flat = flatten_record(item)
87             if flat:
88                 out.write(json.dumps(flat, ensure_ascii=False) + "\n")
89                 saved_count += 1
90
91     print(f"\nProcessing Complete.")
92     print(f"Total Raw Records: {total}")
93     print(f"Saved Valid Records: {saved_count}")
94     print(f"Skipped (Empty/Invalid): {total - saved_count}")
95     print(f"Output File: {OUT_PATH}")
96
97 if __name__ == "__main__":
98     process_all()

```

```

1 # src/build_index.py
2 """
3 Saved FAISS index to: data/processed/embeddings.faiss
4 Saved metadata to: data/processed/metadata.pkl
5 """
6 import json
7 import pickle
8 from tqdm import tqdm
9 import faiss
10 from sentence_transformers import SentenceTransformer
11
12 DATA_PATH = "data/processed/triviaqa_clean.jsonl"
13 INDEX_PATH = "data/processed/embeddings.faiss"
14 META_PATH = "data/processed/metadata.pkl"
15
16 MODEL_NAME = "all-MiniLM-L6-v2"
17 BATCH_SIZE = 64
18 USE_COSINE = True
19
20 def main():
21     model = SentenceTransformer(MODEL_NAME)
22
23     texts = []
24     doc_id_strs = []
25     question_ids = []
26     questions = []
27     answers = []
28     ranks = []
29     urls = []
30
31     # Read question-level JSONL, flatten into passage-level lists
32     with open(DATA_PATH, "r", encoding="utf-8") as f:
33         for line in tqdm(f, desc="Reading processed data"):
34             item = json.loads(line)
35
36             qid = item.get("id", "")
37             q = item.get("question", "")
38             a = item.get("answer", "")
39
40             if not qid or not q or not a:
41                 continue
42
43             for d in item.get("docs", []):
44                 text = d.get("text", "")

```

```

45         if not text:
46             continue
47
48         rank = d.get("rank", None)
49         try:
50             rank_int = int(rank) if rank is not None else None
51         except Exception:
52             rank_int = None
53
54         # Stable doc id (needed for Layer-1 cache keys)
55         faiss_id = len(texts)
56         doc_id_str = f"{qid}_{rank_int}" if rank_int is not None else f"{qid}_{faiss_id}"
57
58         texts.append(text)
59         doc_id_strs.append(doc_id_str)
60         question_ids.append(qid)
61         questions.append(q)
62         answers.append(a)
63         ranks.append(rank_int)
64         urls.append(d.get("url", ""))
65
66     if len(texts) == 0:
67         raise RuntimeError("No passages found. Check preprocess output format and DATA_PATH.")
68
69     print(f"Loaded {len(texts):,} passages. Computing embeddings...")
70
71     embeddings = model.encode(
72         texts,
73         batch_size=BATCH_SIZE,
74         show_progress_bar=True,
75         convert_to_numpy=True,
76         normalize_embeddings=USE_COSINE,
77     ).astype("float32")
78
79     dim = int(embeddings.shape[1])
80     index = faiss.IndexFlatIP(dim) if USE_COSINE else faiss.IndexFlatL2(dim)
81     index.add(embeddings)
82     faiss.write_index(index, INDEX_PATH)
83
84     meta = {
85         "texts": texts,
86         "passages": texts, # Alias for older scripts
87         "question_ids": question_ids,
88         "questions": questions,
89         "answers": answers,
90         "ranks": ranks,
91         "urls": urls,
92         "doc_ids_str": doc_id_strs, # Do NOT use these as L1 keys
93         "model_name": MODEL_NAME,
94         "use_cosine": USE_COSINE,
95         "faiss_dim": dim,
96         "num_vectors": int(index.ntotal),
97     }
98
99     with open(META_PATH, "wb") as f:
100         pickle.dump(meta, f)
101
102     print(f"Saved FAISS index to: {INDEX_PATH}")
103     print(f"Saved metadata to: {META_PATH}")
104     print(f"Index dim={dim}, vectors={index.ntotal:,}")
105
106 if __name__ == "__main__":
107     main()

```

```

1 # src/calibrate_entropy.py
2 # Calibrate entropy thresholds for retrieval confidence estimation
3 """
4 Output:
5 Samples used: 11260
6 H_norm: mean=0.907, std=0.084
7 k_needed: mean=4.54
8
9 Suggested H_norm thresholds (quantiles):

```

```

10 q=0.25 -> H_norm=0.880
11 q=0.50 -> H_norm=0.929
12 q=0.75 -> H_norm=0.963
13 Bucket 1 (-1.000,0.880] count=2815 mean k_needed=4.56
14 Bucket 2 (0.880,0.929] count=2815 mean k_needed=4.38
15 Bucket 3 (0.929,0.963] count=2815 mean k_needed=4.30
16 Bucket 4 (0.963,1.100] count=2815 mean k_needed=4.92
17 """
18 import json
19 import pickle
20 import numpy as np
21 import faiss
22 from tqdm import tqdm
23 from sentence_transformers import SentenceTransformer
24
25 DATA_PATH = "data/processed/triviaqa_clean.jsonl" # Dataset (11260 samples)
26 INDEX_PATH = "data/processed/embeddings.faiss" # FAISS index of passage embeddings
27 META_PATH = "data/processed/metadata.pkl"
28
29 MODEL_NAME = "all-MiniLM-L6-v2" # Sentence-Transformer
30
31 K_MAX = 8
32 TEMPERATURE = 0.1 # Softmax temperature
33 N_SAMPLES = None # None for full dataset
34
35 def softmax(x: np.ndarray, tau: float) -> np.ndarray:
36     x = np.asarray(x, dtype=np.float32)
37     x = x / max(float(tau), 1e-8)
38     x = x - np.max(x)
39     ex = np.exp(x)
40     return ex / (np.sum(ex) + 1e-12)
41
42 def entropy_from_scores(scores: np.ndarray, tau: float) -> tuple[float, float]:
43     # For cosine/IP, higher score = better, so use scores directly as logits
44     p = softmax(scores, tau)
45     p = np.clip(p, 1e-12, 1.0)
46     H = float(-np.sum(p * np.log(p)))
47     H_norm = float(H / np.log(len(p))) if len(p) > 1 else 0.0
48     return H, H_norm
49
50 def contains_answer(texts: list[str], ans: str) -> bool:
51     a = (ans or "").lower().strip()
52     if not a:
53         return False
54     return any(a in (t or "").lower() for t in texts)
55
56 def main():
57     model = SentenceTransformer(MODEL_NAME)
58     index = faiss.read_index(INDEX_PATH)
59
60     with open(META_PATH, "rb") as f:
61         meta = pickle.load(f)
62
63     texts = meta.get("passages", meta.get("texts"))
64     if texts is None:
65         raise KeyError("metadata.pkl must contain 'texts' or 'passages'")
66
67     use_cosine = bool(meta.get("use_cosine", True))
68
69     # (H_norm, best_k_needed) pairs
70     pairs = []
71     total = 0
72
73     with open(DATA_PATH, "r", encoding="utf-8") as f:
74         for i, line in enumerate(tqdm(f, desc="Calibrating")):
75             if N_SAMPLES is not None and i >= N_SAMPLES:
76                 break
77
78             item = json.loads(line)
79             q = item["question"]
80             ans = item["answer"]
81             if not q:
82                 continue

```

```

84     q_emb = model.encode(
85         [q],
86         convert_to_numpy=True,
87         normalize_embeddings=use_cosine
88     ).astype("float32")
89
90     scores, idxs = index.search(q_emb, K_MAX)
91     scores = scores[0]
92     idxs = idxs[0]
93
94     valid = idxs >= 0
95     scores = scores[valid]
96     idxs = idxs[valid]
97     if idxs.size == 0:
98         continue
99
100     H, Hn = entropy_from_scores(scores, TEMPERATURE)
101
102     # Find smallest k that retrieves answer (proxy)
103     top_texts = [texts[int(j)] for j in idxs]
104     best_k = None
105     for k in range(1, len(top_texts) + 1):
106         if contains_answer(top_texts[:k], ans):
107             best_k = k
108             break
109     if best_k is None:
110         best_k = len(top_texts)
111
112     pairs.append((Hn, float(best_k)))
113     total += 1
114
115 pairs = np.asarray(pairs, dtype=np.float32)
116 Hn = pairs[:, 0]
117 Kneed = pairs[:, 1]
118
119 print(f"\nSamples used: {total}")
120 print(f"H_norm: mean={Hn.mean():.3f}, std={Hn.std():.3f}")
121 print(f"k_needed: mean={Kneed.mean():.2f}")
122
123 # Simple thresholding by quantiles (3 buckets example)
124 qs = [0.25, 0.50, 0.75]
125 thr = np.quantile(Hn, qs).tolist()
126 print("\nSuggested H_norm thresholds (quantiles):")
127 for qv, tv in zip(qs, thr):
128     print(f"q={qv:.2f} -> H_norm={tv:.3f}")
129
130 # Show mean k_needed per bucket
131 buckets = [(-1, thr[0]), (thr[0], thr[1]), (thr[1], thr[2]), (thr[2], 1.1)]
132 for bi, (lo, hi) in enumerate(buckets):
133     mask = (Hn > lo) & (Hn <= hi)
134     if mask.sum() == 0:
135         continue
136     print(
137         f"Bucket {bi+1} ({lo:.3f},{hi:.3f}) "
138         f"count={int(mask.sum())} mean k_needed={Kneed[mask].mean():.2f}"
139     )
140
141 if __name__ == "__main__":
142     main()

```

```

1 # src/test_entropy.py
2 """
3 Output:
4 --- Entropy OFF (fixed k=8) ---
5 Accuracy proxy: 7005/11260 = 62.21%
6 Avg retrieved tokens: 280.86
7 Avg embed ms: 9.945
8 Avg FAISS retr ms: 9.747
9
10 --- Entropy ON (adaptive k) ---
11 Accuracy proxy: 6612/11260 = 58.72%
12 Avg k: 6.00
13 Avg retrieved tokens: 210.78

```



```

14 Avg embed ms: 9.945
15 Avg FAISS retr ms: 9.747
16
17 --- Comparison ---
18 Accuracy delta (pp): -3.49
19 Token reduction% (Gain/Loss): +24.95%
20
21 --- Adaptive-k distribution ---
22 k= 4: count= 2816 hit-rate= 53.73%
23 k= 5: count= 2823 hit-rate= 60.33%
24 k= 7: count= 2808 hit-rate= 64.71%
25 k= 8: count= 2813 hit-rate= 56.13%
26 """
27 import argparse
28 import json
29 import pickle
30 import time
31 import numpy as np
32 import faiss
33 from collections import defaultdict
34 from tqdm import tqdm
35 from sentence_transformers import SentenceTransformer
36
37 # Paths
38 DATA_PATH = "data/processed/triviaqa_clean.jsonl"
39 INDEX_PATH = "data/processed/embeddings.faiss"
40 META_PATH = "data/processed/metadata.pkl"
41
42 MODEL_NAME = "all-MiniLM-L6-v2"
43 K_MAX = 8
44 TAU = 0.1
45 DEFAULT_THR = (0.880, 0.929, 0.963)
46 DEFAULT_KS = (4, 5, 7, 8)
47
48 def softmax(x: np.ndarray, tau: float) -> np.ndarray:
49     x = np.asarray(x, dtype=np.float32)
50     tau = max(float(tau), 1e-8)
51     x = x / tau
52     x = x - np.max(x)
53     ex = np.exp(x)
54     return ex / (np.sum(ex) + 1e-12)
55
56 def compute_entropy_norm(scores: np.ndarray, tau: float = TAU, higher_is_better: bool = True) -> float:
57     s = np.asarray(scores, dtype=np.float32)
58     if s.size == 0:
59         return 0.0
60     logits = s if higher_is_better else (-s)
61     p = softmax(logits, tau)
62     p = np.clip(p, 1e-12, 1.0)
63     H = float(-np.sum(p * np.log(p)))
64     return float(H / np.log(len(p))) if len(p) > 1 else 0.0
65
66 def adaptive_k(H_norm: float, thr=DEFAULT_THR, ks=DEFAULT_KS) -> int:
67     t1, t2, t3 = thr
68     if H_norm <= t1: return int(ks[0])
69     if H_norm <= t2: return int(ks[1])
70     if H_norm <= t3: return int(ks[2])
71     return int(ks[3])
72
73 def approx_tokens(text: str) -> int:
74     return max(1, len((text or "").split()))
75
76 def contains_answer_in_docs(docs_text: list[str], ans: str) -> bool:
77     a = (ans or "").lower().strip()
78     if not a:
79         return False
80     return any(a in (t or "").lower() for t in docs_text)
81
82 def main():
83     ap = argparse.ArgumentParser()
84     ap.add_argument("--limit", type=int, default=None,
85                     help="Limit number of dataset questions (default: full).")
86     args = ap.parse_args()
87

```

```

88 thr = DEFAULT_THR
89 ks = DEFAULT_KS
90 kmax = K_MAX
91 tau = TAU
92
93 print("Loading model / index / metadata...")
94 model = SentenceTransformer(MODEL_NAME)
95 index = faiss.read_index(INDEX_PATH)
96 with open(META_PATH, "rb") as f:
97     meta = pickle.load(f)
98 passages = meta.get("passages", meta.get("texts"))
99 if passages is None:
100     raise KeyError("metadata.pkl must contain 'texts' or 'passages'")
101
102 n = 0
103 # OFF (fixed k=8)
104 off_hits = 0
105 off_tokens = 0
106 off_embed_ms = 0.0
107 off_retr_ms = 0.0
108 # ON (adaptive k)
109 on_hits = 0
110 on_tokens = 0
111 on_embed_ms = 0.0
112 on_retr_ms = 0.0
113 on_k_sum = 0
114 H_list = []
115
116 # Bucket stats for ON
117 bucket_counts = defaultdict(int)
118 bucket_hit_sum = defaultdict(int)
119
120 with open(DATA_PATH, "r", encoding="utf-8") as f:
121     for line in tqdm(f):
122         if args.limit is not None and n >= args.limit:
123             break
124         item = json.loads(line)
125         q = item.get("question", "")
126         ans = item.get("answer", "")
127         if not q or not ans:
128             continue
129
130         # 1. Encode timing
131         t0 = time.time()
132         q_emb = model.encode([q], convert_to_numpy=True, normalize_embeddings=True).astype("float32")
133         embed_ms = (time.time() - t0) * 1000.0
134
135         # 2. FAISS retrieval timing (single search used for both OFF/ON)
136         t1 = time.time()
137         scores, ids = index.search(q_emb, kmax)
138         retr_ms = (time.time() - t1) * 1000.0
139
140         scores = scores[0]
141         ids = ids[0]
142         valid = ids >= 0
143         scores = scores[valid]
144         ids = ids[valid]
145         if ids.size == 0:
146             continue
147
148         # 3. Entropy + adaptive k
149         Hn = compute_entropy_norm(scores, tau=tau, higher_is_better=True)
150         k_adapt = adaptive_k(Hn, thr=thr, ks=ks)
151
152         # Entropy OFF: fixed kmax
153         ids_off = ids[:kmax].tolist()
154         docs_off = [passages[int(i)] for i in ids_off]
155         tok_off = sum(approx_tokens(t) for t in docs_off)
156         hit_off = 1 if contains_answer_in_docs(docs_off, ans) else 0
157
158         # Entropy ON: adaptive k
159         ids_on = ids[:k_adapt].tolist()
160         docs_on = [passages[int(i)] for i in ids_on]
161         tok_on = sum(approx_tokens(t) for t in docs_on)

```

```

162         hit_on = 1 if contains_answer_in_docs(docs_on, ans) else 0
163
164         # Aggregates
165         n += 1
166         off_hits += hit_off
167         off_tokens += tok_off
168         off_embed_ms += embed_ms
169         off_retr_ms += retr_ms
170         on_hits += hit_on
171         on_tokens += tok_on
172         on_embed_ms += embed_ms
173         on_retr_ms += retr_ms
174         on_k_sum += k_adapt
175         H_list.append(Hn)
176
177         # Bucket logging
178         bucket_key = k_adapt
179         bucket_counts[bucket_key] += 1
180         bucket_hit_sum[bucket_key] += hit_on
181
182     if n == 0:
183         raise RuntimeError("No valid samples processed. Check DATA_PATH format.")
184
185     def pct(x): return 100.0 * x
186
187     # Print results
188     print("\n--- Entropy OFF (fixed k=8) ---")
189     print(f"Accuracy proxy: {off_hits}/{n} = {pct(off_hits/n):.2f}%")
190     print(f"Avg retrieved tokens: {off_tokens/n:.2f}")
191     print(f"Avg embed ms: {off_embed_ms/n:.3f}")
192     print(f"Avg FAISS retr ms: {off_retr_ms/n:.3f}")
193
194     print("\n--- Entropy ON (adaptive k) ---")
195     print(f"Accuracy proxy: {on_hits}/{n} = {pct(on_hits/n):.2f}%")
196     print(f"Avg k: {on_k_sum/n:.2f}")
197     print(f"Avg retrieved tokens: {on_tokens/n:.2f}")
198     print(f"Avg embed ms: {on_embed_ms/n:.3f}")
199     print(f"Avg FAISS retr ms: {on_retr_ms/n:.3f}")
200
201     print("\n--- Comparison ---")
202     print(f"Accuracy delta (pp): {pct(on_hits/n)-pct(off_hits/n):+.2f}")
203     if off_tokens > 0:
204         token_red_pct = pct((off_tokens - on_tokens) / off_tokens)
205     else:
206         token_red_pct = 0.0
207     print(f"Token reduction% (Gain/Loss): {token_red_pct:+.2f}%")
208
209     print("\n--- Adaptive-k distribution ---")
210     for k in sorted(bucket_counts.keys()):
211         c = bucket_counts[k]
212         hr = bucket_hit_sum[k] / c if c else 0.0
213         print(f"k={k:2d}: count={c:6d} hit-rate={pct(hr):6.2f}%")
214
215 if __name__ == "__main__":
216     main()

```

```

1 # src/simulator.py
2 """
3 RAGCache (L1) + Layer2 semantic reuse Simulator
4 =====
5 Core simulation architecture is:
6 1. Retrieval Entropy (How many docs do we actually need?)
7 2. L1 Cache (Have we seen this sequence of docs before?)
8 3. L2 Cache (Is this query semantically similar to a previous one?)
9 """
10 import time
11 import numpy as np
12 from dataclasses import dataclass, field
13 from typing import List, Tuple, Dict, Any
14
15 # -----
16 # 1. ENTROPY (Shannon)
17 # -----

```

```

18 def softmax(x: np.ndarray, tau: float) -> np.ndarray:
19     # P(i) = exp(score_i / tau) / sum(exp(score_j / tau))
20     x = np.asarray(x, dtype=np.float32)
21     tau = max(float(tau), 1e-8)
22     x = x / tau
23     x = x - np.max(x)
24     ex = np.exp(x)
25     return ex / (np.sum(ex) + 1e-12)
26
27 def compute_entropy_norm(scores: np.ndarray, tau: float = 0.1, higher_is_better: bool = True) -> float:
28     # cosine/IP: higher is better -> logits = scores
29     # L2 distance: lower is better -> logits = -scores
30     s = np.asarray(scores, dtype=np.float32)
31     if s.size == 0: return 0.0
32
33     logits = s if higher_is_better else (-s)
34     p = softmax(logits, tau)
35     p = np.clip(p, 1e-12, 1.0)
36     H = float(-np.sum(p * np.log(p)))
37
38     # Normalize to [0,1] by dividing with log(k)
39     H_norm = float(H / np.log(len(p))) if len(p) > 1 else 0.0
40     return H_norm
41
42 def adaptive_k(H_norm: float,
43               thr: Tuple[float, float, float] = (0.880, 0.929, 0.963),
44               ks: Tuple[int, int, int, int] = (4, 5, 7, 8)) -> int:
45     # Thresholds derived from "src/calibrate_entropy.py" quantiles
46     t1, t2, t3 = thr
47     if H_norm <= t1: return ks[0]
48     if H_norm <= t2: return ks[1]
49     if H_norm <= t3: return ks[2]
50     return ks[3]
51
52 def approx_tokens(t: str) -> int:
53     return max(1, len(t.split()))
54
55 # -----
56 # 2. L1 CACHE SIMULATOR (RAGCache Logic)
57 # -----
58
59 @dataclass
60 class TreeEntry:
61     token_sum: int
62     last_access_ts: float = field(default_factory=time.time)
63
64 class KVCacheSim:
65     """
66     Simulates RAGCache (Prefix Caching).
67     Tracks exact sequences of document IDs.
68     """
69     def __init__(self, compute_ms_per_token: float = 0.05):
70         self.tree: Dict[Tuple[int, ...], TreeEntry] = {}
71         self.compute_ms = float(compute_ms_per_token)
72         self.hits = 0
73         self.misses = 0
74
75     # Finds length of the longest cached prefix.
76     def longest_prefix_hit(self, doc_ids: List[int]) -> int:
77         for i in range(len(doc_ids), 0, -1):
78             if tuple(doc_ids[:i]) in self.tree:
79                 return i
80         return 0
81
82     # Calculates the Time-To-First-Token (Latency) by (Tokens_To_Compute * ms_per_token)
83     def prefill_ttft_ms(self, doc_ids: List[int], token_sizes: List[int], prompt_tokens: int = 32):
84         total_tokens = int(prompt_tokens + sum(token_sizes))
85
86         # 1. Baseline cost
87         base_cost = total_tokens * self.compute_ms
88
89         # 2. Check cache
90         hit_len = self.longest_prefix_hit(doc_ids)
91

```

```

92     # 3. Calculate final cost
93     cached_tokens = int(sum(token_sizes[:hit_len]))
94     remaining_tokens = total_tokens - cached_tokens
95     final_cost = remaining_tokens * self.compute_ms
96
97     if hit_len > 0:
98         self.hits += 1
99         where = "L1"
100    else:
101        self.misses += 1
102        where = "NONE"
103
104    meta = {"where": where, "cached_tokens": cached_tokens, "hit_len_docs": hit_len}
105    return base_cost, final_cost, meta
106
107    # Saves the current sequence into the Knowledge Tree.
108    def insert_path(self, doc_ids: List[int], token_sizes: List[int]):
109        running_key = []
110        token_sum = 0
111        for d, sz in zip(doc_ids, token_sizes):
112            running_key.append(d)
113            token_sum += sz
114            key = tuple(running_key)
115            if key not in self.tree:
116                self.tree[key] = TreeEntry(token_sum=token_sum)
117            self.tree[key].last_access_ts = time.time()
118
119    def stats(self) -> Dict[str, Any]:
120        total = self.hits + self.misses
121        return {
122            "l1_hits": self.hits,
123            "l1_misses": self.misses,
124            "l1_hit_rate": (self.hits / total) if total else 0.0,
125            "l1_num_cached_prefixes": len(self.tree),
126        }
127
128    # -----
129    # 3. L2 CACHE SIMULATOR (Semantic Reuse)
130    # -----
131
132    class Layer2KVAdapter:
133        """
134        Simulates Semantic Reuse.
135        If current retrieved-doc context is similar to a previous one, we assume we can
136        reuse a fraction (reuse_ratio) of the document-prefill KV states.
137        That means we skip reuse_ratio * sum(token_sizes) tokens worth of prefill compute.
138        """
139        def __init__(self, model, sim_threshold: float = 0.85, reuse_ratio: float = 0.6):
140            self.model = model
141            self.sim_th = float(sim_threshold)
142            self.reuse_ratio = float(reuse_ratio)
143            self.protos: List[np.ndarray] = [] # Normalized vectors
144            self.accepts = 0
145            self.rejects = 0
146
147        def _encode_norm(self, docs_text: List[str]) -> np.ndarray:
148            # Encode each document separately
149            embs = self.model.encode(
150                docs_text,
151                convert_to_numpy=True,
152                normalize_embeddings=True
153            ).astype("float32")
154            v = embs.mean(axis=0)
155            v = v / (np.linalg.norm(v) + 1e-12)
156            return v
157
158        def try_reuse(self, docs_text: List[str], token_sizes: List[int]) -> Tuple[bool, Dict[str, Any]]:
159            if not self.protos:
160                self.rejects += 1
161                return False, {"reason": "no_history", "sim": None, "skip_tokens": 0}
162
163            # Encode the current document set context and compare
164            q = self._encode_norm(docs_text)
165            best_sim = float(max(np.dot(q, p) for p in self.protos))

```

```

166
167     # Decision
168     if best_sim >= self.sim_th:
169         self.accepts += 1
170         doc_tokens = int(sum(token_sizes))
171         skip_tokens = int(self.reuse_ratio * doc_tokens)
172         return True, {"reason": "accepted", "sim": best_sim, "skip_tokens": skip_tokens}
173     else:
174         self.rejects += 1
175         return False, {"reason": "low_sim", "sim": best_sim, "skip_tokens": 0}
176
177     def update_proto(self, docs_text: List[str]):
178         self.protos.append(self._encode_norm(docs_text))
179
180     def stats(self) -> Dict[str, Any]:
181         total = self.accepts + self.rejects
182         return {
183             "l2_accepts": self.accepts,
184             "l2_rejects": self.rejects,
185             "l2_accept_rate": (self.accepts / total) if total else 0.0,
186             "l2_num_protos": len(self.protos),
187             "reuse_ratio": self.reuse_ratio,
188             "sim_threshold": self.sim_th,
189         }

```

```

1 # src/query.py
2 import os, json, time, pickle
3 import numpy as np
4 from tqdm import tqdm
5 import faiss
6 from sentence_transformers import SentenceTransformer, util
7 from rapidfuzz import fuzz
8 from simulator import (compute_entropy_norm, adaptive_k, approx_tokens, KVCacheSim, Layer2KVAdapter)
9
10 # Configurations
11 INDEX_PATH = "data/processed/embeddings.faiss"
12 META_PATH = "data/processed/metadata.pkl"
13 DATA_PATH = "data/processed/triviaqa_clean.jsonl"
14 CACHE_FILE = "data/cache/cache_state.pkl"
15
16 BOOTSTRAP_N = 10
17 MODEL_NAME = "all-MiniLM-L6-v2"
18 TOP_K_MAX = 8
19 TAU = 0.1
20 FUZZY_THRESHOLD = 80
21 L2_SIM_TH = 0.90
22 L2_REUSE_RATIO = 0.6
23
24 def percent_gain(base, final):
25     return 0.0 if base <= 1e-9 else 100.0 * (base - final) / base
26
27 def compute_accuracy(answer, docs):
28     if not answer:
29         return None
30     a = answer.lower()
31     for d in docs:
32         if fuzz.partial_ratio(a, d.lower()) >= FUZZY_THRESHOLD:
33             return 1
34     return 0
35
36 # Bootstrap cache (L1+L2) for both situations: Entropy OFF (k=8) and Entropy ON (adaptive k)
37 def bootstrap_caches(model, index, passages, data):
38     kv_off, kv_on = KVCacheSim(), KVCacheSim()
39     l2_off = Layer2KVAdapter(model, sim_threshold=L2_SIM_TH, reuse_ratio=L2_REUSE_RATIO)
40     l2_on = Layer2KVAdapter(model, sim_threshold=L2_SIM_TH, reuse_ratio=L2_REUSE_RATIO)
41
42     for q, _ in tqdm(data, desc="Warming caches"):
43         q_emb = model.encode([q], convert_to_numpy=True, normalize_embeddings=True).astype("float32")
44         scores, ids = index.search(q_emb, TOP_K_MAX)
45         scores = scores[0]
46         ids = ids[0]
47
48     # Entropy for IP/cosine: higher_is_better = True
49     Hn = compute_entropy_norm(scores, tau=TAU, higher_is_better=True)

```

```

50     k = adaptive_k(Hn)
51
52     # Entropy OFF world always uses k=8
53     doc_ids_off = ids[:TOP_K_MAX].tolist()
54     docs_off = [passages[i] for i in doc_ids_off]
55     sizes_off = [approx_tokens(d) for d in docs_off]
56     kv_off.prefill_ttft_ms(doc_ids_off, sizes_off)
57     kv_off.insert_path(doc_ids_off, sizes_off)
58     l2_off.update_proto(docs_off)
59
60     # Entropy ON world uses adaptive k
61     doc_ids_on = ids[:k].tolist()
62     docs_on = [passages[i] for i in doc_ids_on]
63     sizes_on = [approx_tokens(d) for d in docs_on]
64     kv_on.prefill_ttft_ms(doc_ids_on, sizes_on)
65     kv_on.insert_path(doc_ids_on, sizes_on)
66     l2_on.update_proto(docs_on)
67
68     return kv_off, kv_on, l2_off, l2_on
69
70
71 # Load model, FAISS, data
72 print("Loading model and FAISS index...")
73 model = SentenceTransformer(MODEL_NAME)
74 index = faiss.read_index(INDEX_PATH)
75
76 with open(META_PATH, "rb") as f:
77     meta = pickle.load(f)
78
79 passages = meta.get("passages", meta["texts"])
80
81 data = []
82 with open(DATA_PATH, "r", encoding="utf-8") as f:
83     for i, line in enumerate(f):
84         if i >= BOOTSTRAP_N:
85             break
86         item = json.loads(line)
87         q = item["question"]
88         a = item.get("answer", "")
89         data.append((q, a))
90
91
92 # Load / Warmup
93 if os.path.exists(CACHE_FILE):
94     with open(CACHE_FILE, "rb") as f:
95         state = pickle.load(f)
96         kv_off = state["kv_off"]; kv_on = state["kv_on"]
97         l2_off = state["l2_off"]; l2_on = state["l2_on"]
98         print(f"Loaded caches from {CACHE_FILE}")
99 else:
100     kv_off, kv_on, l2_off, l2_on = bootstrap_caches(model, index, passages, data)
101     with open(CACHE_FILE, "wb") as f:
102         pickle.dump(
103             dict(kv_off=kv_off, kv_on=kv_on, l2_off=l2_off, l2_on=l2_on),
104             f
105         )
106     print(f"Bootstrapped caches and saved to {CACHE_FILE}")
107
108
109 # Dataset embeddings for nearest answer (accuracy proxy)
110 ds_qs, ds_as = zip(*data)
111 ds_emb = model.encode(list(ds_qs), convert_to_numpy=True, normalize_embeddings=True).astype("float32")
112
113 def nearest_dataset_answer(q, sim_th=0.8):
114     q_emb = model.encode([q], convert_to_numpy=True, normalize_embeddings=True).astype("float32")
115     sims = util.cos_sim(q_emb, ds_emb)[0].cpu().numpy()
116     j = int(np.argmax(sims))
117     return ds_as[j] if sims[j] >= sim_th else None
118
119
120 # Retrieval runner (one world)
121 def run_world(q, use_entropy, kv, l2):
122     q_emb = model.encode([q], convert_to_numpy=True, normalize_embeddings=True).astype("float32")
123

```

```

124 t0 = time.time()
125 scores, ids = index.search(q_emb, TOP_K_MAX)
126 retr_ms = (time.time() - t0) * 1000.0
127
128 scores = scores[0]
129 ids = ids[0]
130
131 if use_entropy:
132     Hn = compute_entropy_norm(scores, tau=TAU, higher_is_better=True)
133     k = adaptive_k(Hn)
134 else:
135     Hn = np.nan
136     k = TOP_K_MAX
137
138 doc_ids = ids[:k].tolist()
139 docs = [passages[i] for i in doc_ids]
140 sizes = [approx_tokens(d) for d in docs]
141
142 base, l1_ms, meta_l1 = kv.prefill_ttfms(doc_ids, sizes)
143 final = l1_ms
144 where = meta_l1["where"]
145
146 l2_meta = {"reason": "not_checked", "sim": None, "skip_tokens": 0}
147 if where == "NONE":
148     ok, l2_meta = l2.try_reuse(docs, sizes)
149     if ok:
150         # L2 means: skip part of doc-prefill compute
151         final = max(0.0, final - l2_meta["skip_tokens"] * kv.compute_ms)
152         where = "L2"
153
154 acc = compute_accuracy(nearest_dataset_answer(q), docs)
155 return dict(
156     Hn=Hn, k=k, retr_ms=retr_ms,
157     base=base, final=final, where=where,
158     acc=acc, l2_meta=l2_meta
159 )
160
161 # Interactive Query Loop
162 print("System ready. Type 'exit' to quit.\n")
163 while True:
164     q = input("Your question: ").strip()
165     if not q or q.lower() in ("exit", "quit", ":q"):
166         break
167
168     # Show entropy for info
169     q_emb = model.encode([q], convert_to_numpy=True, normalize_embeddings=True).astype("float32")
170     tmp_s, _ = index.search(q_emb, TOP_K_MAX)
171     Hn_info = compute_entropy_norm(tmp_s[0], tau=TAU, higher_is_better=True)
172     k_info = adaptive_k(Hn_info)
173     print(f"\nEntropy H_norm = {Hn_info:.3f} -> adaptive k={k_info}")
174
175     off = run_world(q, use_entropy=False, kv=kv_off, l2=l2_off)
176     on = run_world(q, use_entropy=True, kv=kv_on, l2=l2_on)
177
178     g_off = percent_gain(off["base"], off["final"])
179     g_on = percent_gain(on["base"], on["final"])
180
181     def fmt_acc(v): return "NA" if v is None else str(v)
182
183     def fmt_l2(x):
184         m = x["l2_meta"]
185         if x["where"] == "L2":
186             return f"L2 HIT (sim={m['sim']:.3f}, skip_tokens={m['skip_tokens']})"
187         if x["where"] == "NONE":
188             return f"L2 MISS (reason={m['reason']}, sim={m['sim']})"
189         return "L2 not checked"
190
191     print(f"Entropy OFF: path={off['where']:<4} | k=8 | retr={off['retr_ms']:.2f} ms | TTFT gain={g_off:.2f}% | acc={fmt_acc(off['acc'])} | {fmt_l2(off)}")
192     print(f"Entropy ON: path={on['where']:<4} | k={on['k']}>4} | retr={on['retr_ms']:.2f} ms | TTFT gain={g_on:.2f}% | acc={fmt_acc(on['acc'])} | {fmt_l2(on)}\n")
193
194 # Online adaptation: update both caches with their own chosen k
195

```



```
196 q_emb = model.encode([q], convert_to_numpy=True, normalize_embeddings=True).astype("float32")
197 scores, ids = index.search(q_emb, TOP_K_MAX)
198 ids = ids[0].tolist()
199 docs8 = [passages[i] for i in ids]
200 sizes8 = [approx_tokens(d) for d in docs8]
201 kv_off.insert_path(ids, sizes8)
202 l2_off.update_proto(docs8)
203
204 # Entropy ON world stores adaptive-k
205 docsK = docs8[:k_info]
206 sizesK = sizes8[:k_info]
207 kv_on.insert_path(ids[:k_info], sizesK)
208 l2_on.update_proto(docsK)
```